

Langgraph

에이전트를 잘 구축하는 법 - Anthropic

Building effective agents Anthropic

다양한 산업 분야에서 대형 언어 모델(LLM) 에이전트를 구축하는 수십 개의 팀과 함께 한 경험에서,

일관되게, 가장 **성공적인 구현 사례**들은 **복잡한 프레임워크**나 **특화된 라이브러리**를 사용하지 않았으며, 대신 **간단하고 조합 가능한 패턴**을 활용해 구축되었습니다.

효과적인 에이전트를 구축하는 데 도움이 될 실질적인 조언

- 언제 에이전트를 사용하거나 / 사용해서는 안되는지 구분
- 언제, 어떻게 프레임워크를 사용해야 할지

언제, 어떻게 프레임워크를 사용해야 할지

언제 사용해야 할까?

- 빠른 시작 및 프로토타이핑: 작업 단순화에 유리
- 주의: 복잡한 작업들이 숨겨져 있으므로, 디버깅이 어려울 수 있음
- 주의: 프레임워크에서 제공하는 기능을 벗어나는 경우는 불필요한 추상화로 인해 오히려 복잡성이 증가 할 수 있음

어떻게 사용해야 할까?

- 직접 LLM API 사용: 간단한 코드로 다양한 패턴 구현 가능
- 프레임워크 사용 시 내부 코드 이해 필수: 추상화로 인한 오해 방지
- 상황에 맞게 혼합 사용: 단순할 땐 직접, 반복 작업이나 복잡한 경우에는 프레임워크 활용

> 프레임워크에 대한 충분한 이해 없이, 프레임워크에 의존하는 것은 비효율적임

프로젝트 구조

Langgraph Cli

- langgraph new
 - 프로젝트 생성
- langgraph dev
 - 개발 환경
- langgraph build
 - 빌드
- langgraph up
 - 빌드, 배포

실행 및 테스트

스크립트

```
graph = graph_builder.compile()

result = graph.invoke({"topic": "인공지능의 현재와 미래"})

print(result)
```

CLI

```
** LANGSMITH_API_KEY 환경변수 지정 필요

langgraph up
```

Langgraph Studio

```
** Langgraph Studio 설치 필요

** WEB UI
```

Jupyter Notebook

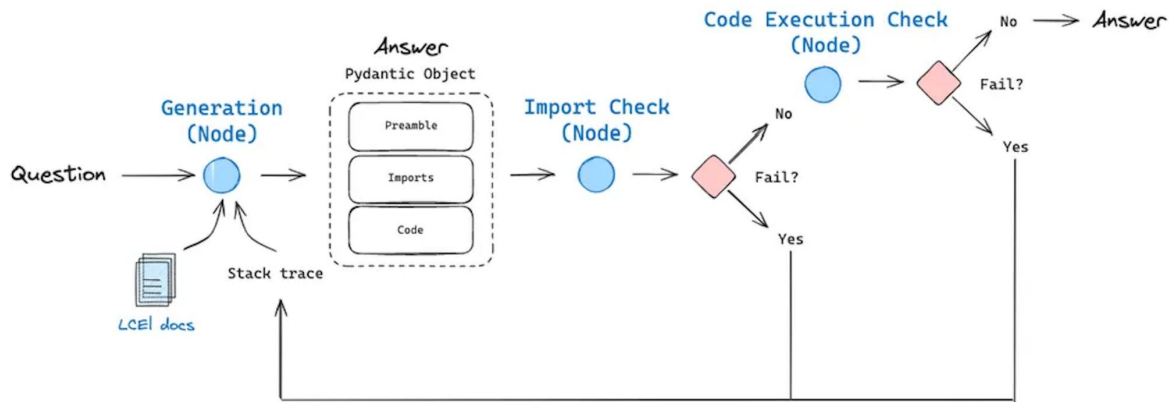
```
display(Image(graph.get_graph().draw_mermaid_png()))
```

Rest API Client

랭그래프 스튜디오 참고

Graph

- State
 - 공유 데이터: TypedDict or Pydantic BaseModel
- Nodes
 - State를 input으로 전달 받음
 - Side effect
 - Return updated State
- Edges
 - Node, Node의 연결



Graph - State

LangGraph's underlying graph algorithm uses message passing to define a general program (https://en.wikipedia.org/wiki/Message_passing)

StateGraph 클래스 (State + Node + Edge)

```
state_graph = StateGraph(MessagesState)
```

```
state_graph.add_node(...)
```

```
state_graph.add_edge(...)
```

```
...
```

```
graph = state_graph.compile() # 그래프 완성
```

Graph - without State

State가 없다면.

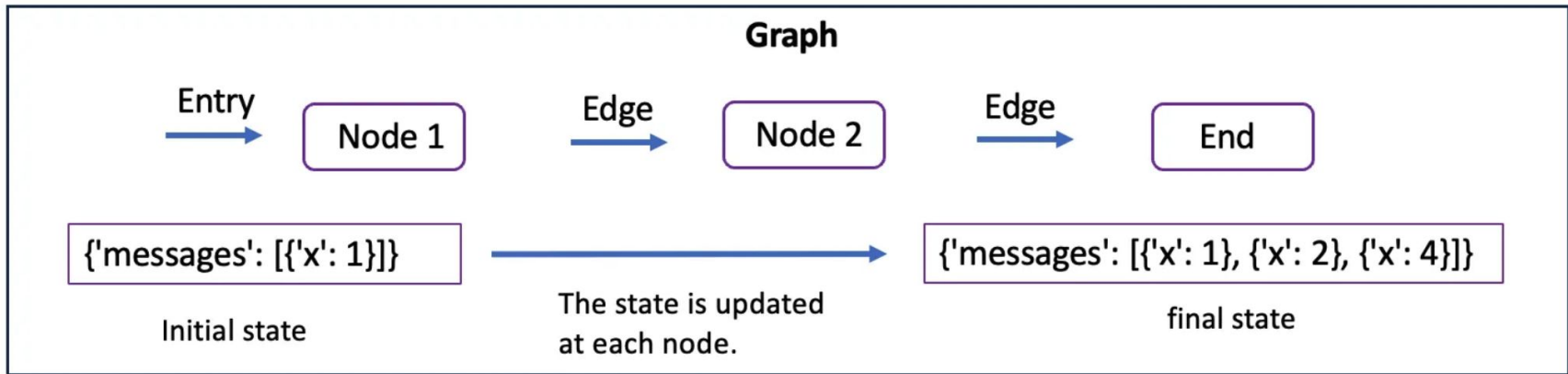
컨텍스트 유지 불가 - LLM이 이전 대화나 데이터를 기억하지 못하고, 매번 새롭게 시작

분기 처리 어려움 - 특정 조건에 따라 실행 흐름을 다르게 하는 것이 어려움

병렬 실행 후 데이터 - 동시성 문제, 합치기 어려움. 여러 개의 결과를 한 번에 받아서 저장할 방법이 없음

비효율적인 코드 구조 - 여러 개의 변수를 글로벌하게 관리해야 하는 경우가 많아짐

Graph - State



- 그래프의 스키마
- 리듀서 (상태를 어떻게 관리할지)
 - 예제) `simple_graph` 의 `add_one`, `smart_agent_simple`의 `add_messages`

Graph - State - Message

대화의 히스토리를 저장하고 관리하는 것은 일반적인 과정이므로

Message 클래스를 제공한다.

- BaseMessage, AnyMessage, HumanMessage, ...

또한 **add_messages** 함수를 제공함으로써 메시지를 처리하는 reducer를 제공한다.

Message 클래스는 serialization / deserialization 기능을 제공한다.

Graph - Node

일반적으로 함수로 구성 되어 있으며,

상태와 **config**에 대한 인자를 받음

```
def my_node(state: dict, config: RunnableConfig) -> state
```

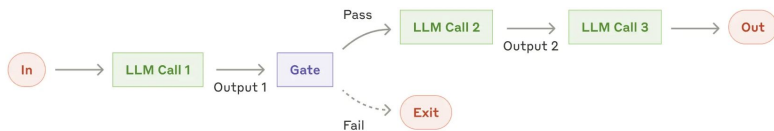
- START Node
- END Node

Graph - Edge

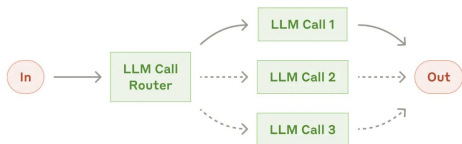
- Normal Edges
 - `graph.add_edge("node_a", "node_b")`
- Conditional Edges
 - `graph.add_conditional_edges("node_a", routing_function)`
 - `graph.add_conditional_edges("node_a", routing_function, {True: "node_b", False: "node_c"})`
- Entry Point
 - `graph.add_edge(START, "node_a")`

Workflows

프롬프트 체이닝 (Prompt Chaining)



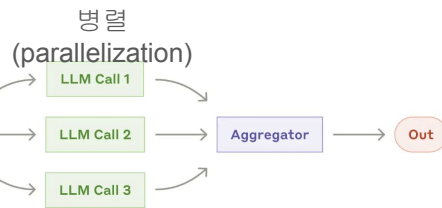
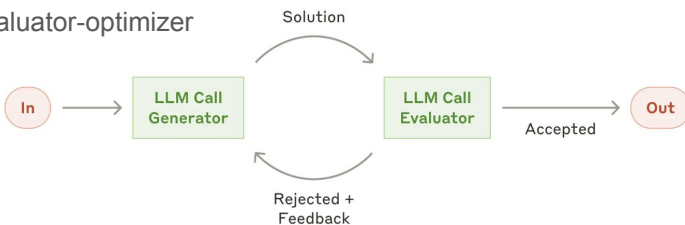
라우팅(Routing)



Orchestrator-workers

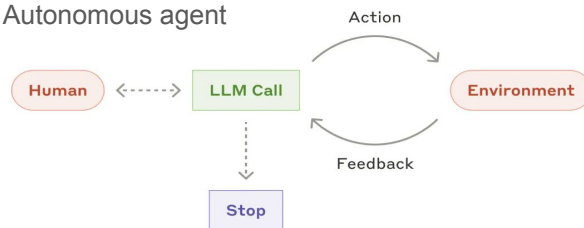


evaluator-optimizer



자율 에이전트

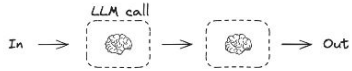
Autonomous agent



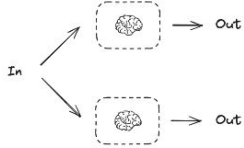
Workflows vs Agent

Workflows

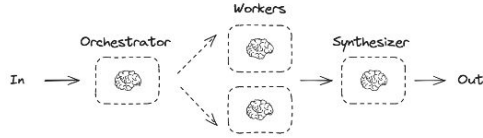
Prompt Chaining



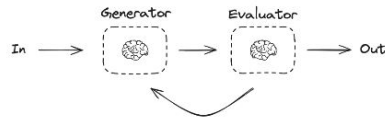
Parallelization



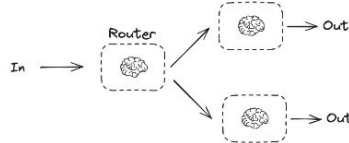
Orchestrator-Worker



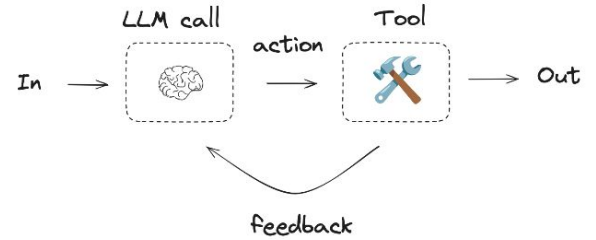
Evaluator-optimizer



Routing



Agent



https://langchain-ai.github.io/langgraph/concepts/img/agent_workflow.png

Workflows vs Agent

Workflows

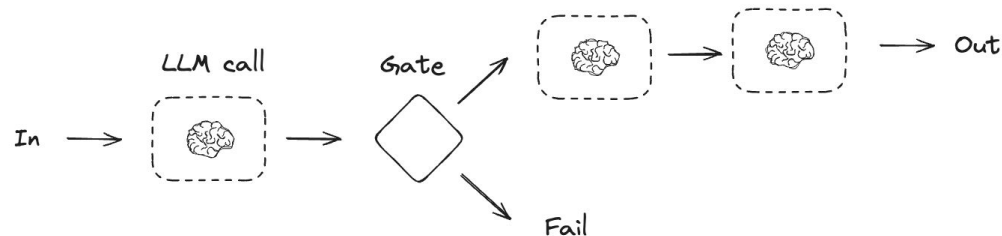
- LLM과 도구들이 미리 정의된 코드 경로를 통해 조율되는 시스템
- 고정된 단계와 명확한 실행 흐름을 가짐
- 예측 가능하고 제어된 방식으로 작업 수행

Agent

- LLM이 동적으로 자신의 프로세스와 도구 사용을 지정
- 작업 수행 방법을 스스로 결정하고 제어
- 더 유연하고 자율적인 의사결정 가능

Workflows - Prompt Chaining

- 정의: 각 LLM 호출이 이전 단계의 출력을 처리하는 연쇄적 구조
- 사용 시기:
 - 작업이 명확한 하위 작업으로 분해 가능할 때
 - 각 단계가 순차적으로 처리되어야 할 때
- 장점:
 - 각 단계를 더 작고 쉬운 작업으로 분해
 - 중간 결과 검증 가능
 - 높은 정확도 달성

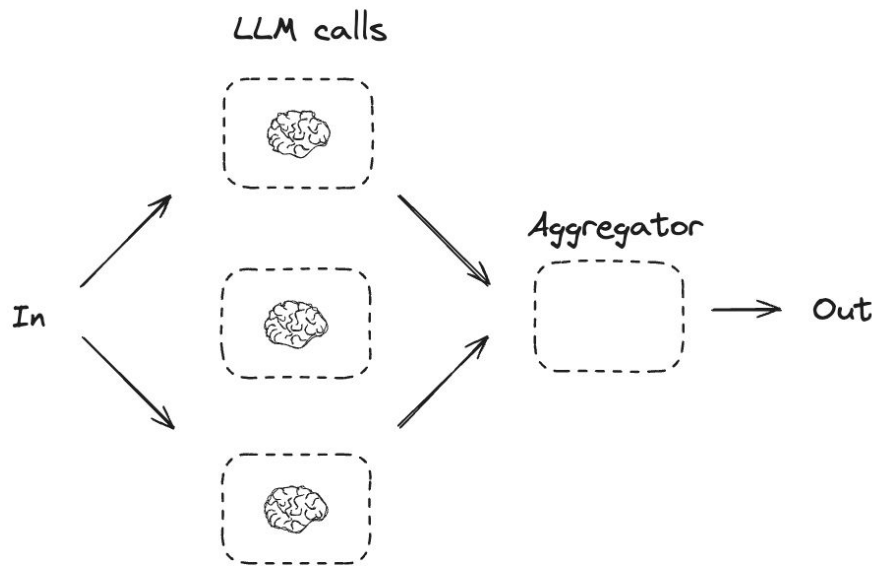


예제: ex-chain

Workflows - 병렬화(Parallelization)

- 정의: 독립적인 작업들을 동시에 실행하는 구조
- 사용 시기:
 - 여러 작업이 서로 독립적일 때
 - 처리 시간 단축이 필요할 때
- 장점:
 - 전체 실행 시간 단축
 - 시스템 리소스 효율적 활용
 - 확장성 향상

예제: `ex_parallelization`



Workflows - 라우팅(Routing) 패턴

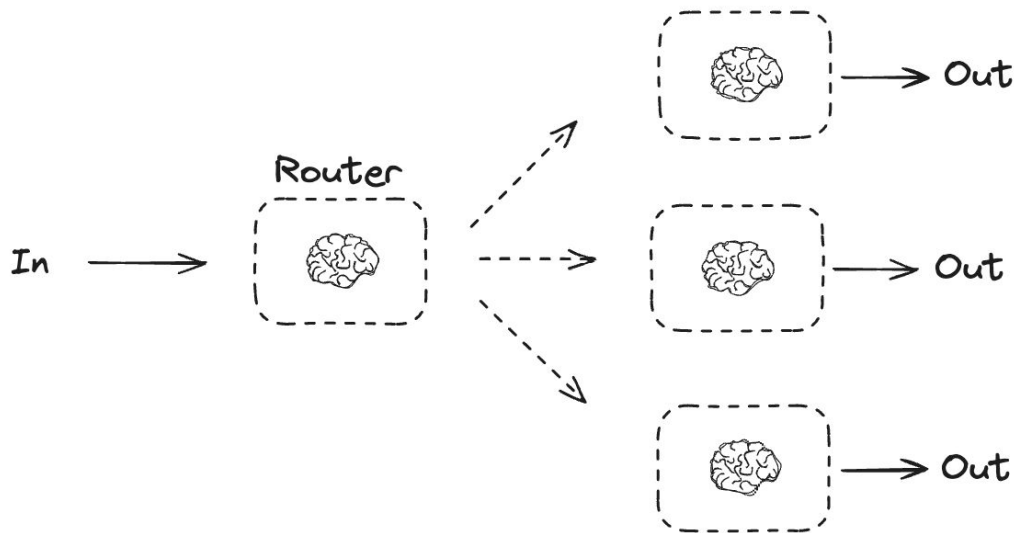
- 정의: 조건에 따라 다른 실행 경로를 선택하는 구조

- 사용 시기:

- 복잡한 의사결정이 필요할 때
- 상황에 따라 다른 처리가 필요할 때

- 장점:

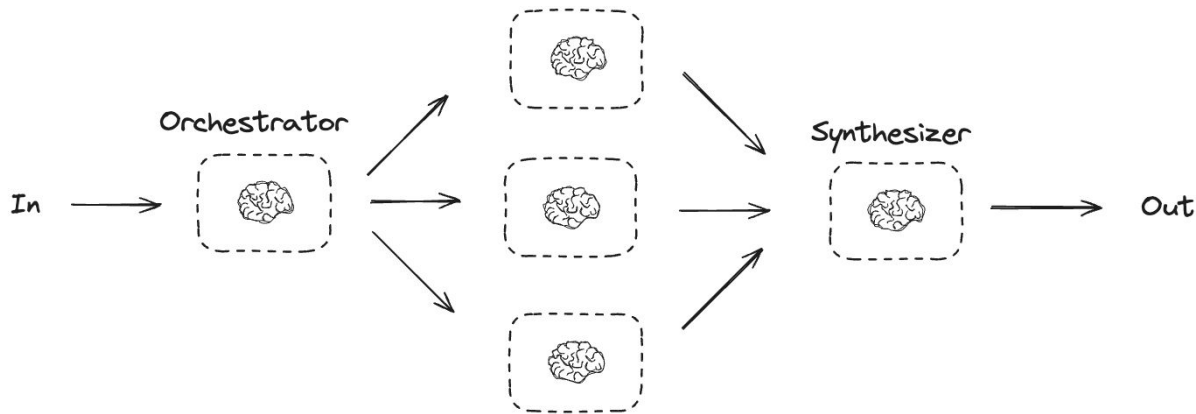
- 유연한 워크플로우 제어
- 조건부 로직 구현
- 동적 경로 선택



예제: `ex_routing`

Workflows - Orchestrator-Worker

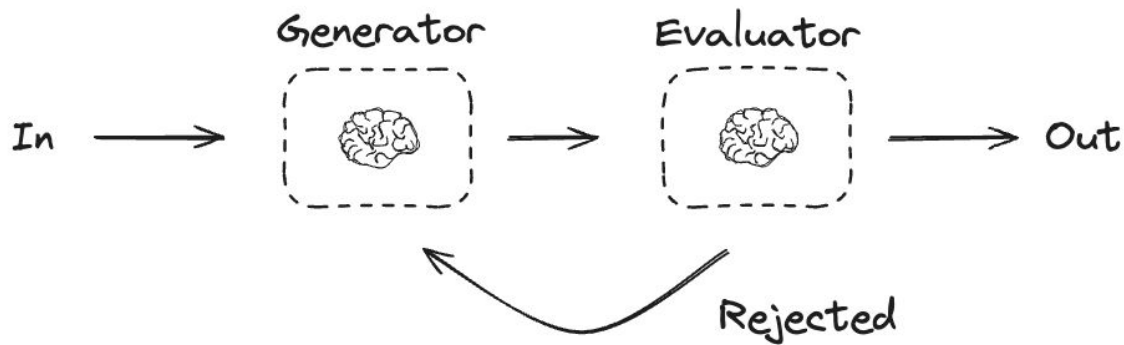
- 정의: 중앙 조정자가 여러 작업자를 관리하는 구조
- 사용 시기:
 - 복잡한 작업 조정이 필요할 때
 - 여러 컴포넌트 간 협업이 필요할 때
- 장점:
 - 중앙 집중식 제어
 - 작업 분배 최적화
 - 확장 가능한 아키텍처



예제: `ex_worker`

Evaluator-optimizer

- 정의: 결과를 평가하고 최적화하는 피드백 루프 구조
- 사용 시기:
 - 결과의 품질 향상이 필요할 때
 - 반복적인 개선이 필요할 때
- 장점:
 - 지속적인 품질 개선
 - 자동화된 최적화
 - 성능 메트릭 추적



예제: `ex_optimizer`

Agent

- 정의: LLM이 환경의 피드백을 기반으로 도구를 사용하여 작업을 수행하는 반복적 구조

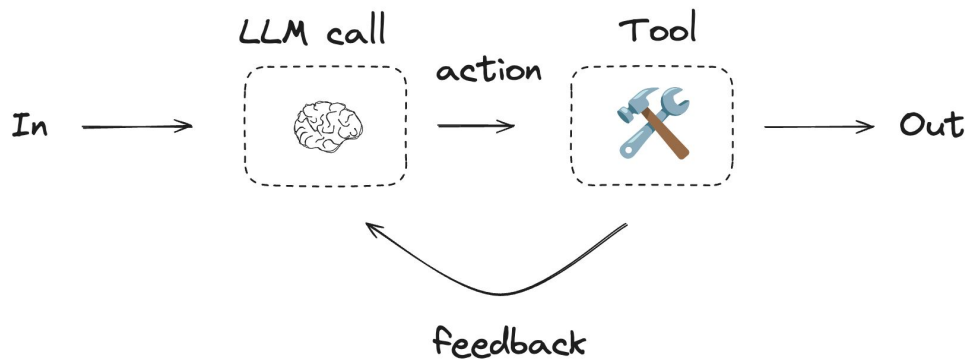
- 사용 시기:

- 단계 수를 예측할 수 없는 열린 문제일 때
- 고정된 경로를 하드코딩할 수 없는 상황
- 여러 턴에 걸쳐 작업을 수행해야 할 때

- LangGraph Agent

- Pre-built Agent (`create_react_agent`)
- 도구 바인딩과 상태 관리
- 메시지 기반 상호작용

예제: `ex_agent`



Autonomous Agent

ReAct (Reasoning + Acting) 패턴

- 정의: 추론과 행동을 반복하며 문제를 해결
- 사용 시기:
 - 복잡한 추론이 필요한 작업
 - 동적인 의사결정이 필요할 때

도구 사용 Agent 패턴

- 정의: LLM이 주어진 도구들을 자율적으로 선택하고 사용
- 사용 시기:
 - 다양한 도구 조합이 필요할 때
 - 상황에 따른 도구 선택이 중요할 때

멀티 Agent 협업 패턴

- 정의: 여러 특화된 Agent들이 협력하여 문제를 해결하는
- 사용 시기:
 - 복잡한 작업을 분할 정복해야 할 때
 - 다양한 전문성이 필요할 때

자기 개선 Agent 패턴

- 정의: Agent가 자신의 성능을 모니터링하고 개선
- 사용 시기:
 - 지속적인 성능 향상이 필요할 때
 - 새로운 상황 적응이 필요할 때

Functional API

비교: <https://langchain-ai.github.io/langgraph/tutorials/workflows/#parallelization>

새로운 API 소개: <https://blog.langchain.dev/introducing-the-langgraph-functional-api/>

제어 흐름

- **Functional API**는 그래프 구조 고민 없이 **Python** 문법 그대로 사용 가능.
- 코드량 줄어듦.

상태 관리

- **Graph API**는 상태 선언과 리듀서 필요할 수도 있음. **Functional API**는 함수 내부에서만 상태 유지, 별도 관리 불필요.

Time Travel (snapshot)

- **Graph API**는 노드 실행마다 체크포인트 생성. **Functional API**는 `entrypoint` 실행 시만 생성. **Graph API**가 더 세밀함.

시각화

- **Graph API**는 워크플로우 시각화 가능해 디버깅, 공유에 유리함. **Functional API**는 실행 흐름이 동적이라 시각화 불가.